

Capitolul 0

Recapitulare detaliată Java

Programare Avansată pe Obiecte în Java

Ghid de învățare cu explicații, exemple, analogii și capcane de examen

Scopul documentului: să ai o bază clară înainte de capitolele grele: moștenire, polimorfism, colecții, excepții, fișiere, lambda, stream-uri și fire de execuție.

Cuprins

Cum folosești documentul	5
1 Imaginea de ansamblu: ce este Java	5
1.1 Compilare și rulare	5
1.2 JDK, JRE, JVM	5
1.3 Analogie ușor de reținut	5
2 Structura minimă a unui program Java	6
2.1 Ce înseamnă fiecare parte	6
3 Pachete și importuri	6
3.1 Reguli importante	6
4 Tipuri de date în Java	7
4.1 Tipuri primitive	7
4.2 Tipuri referință	7
4.3 Analogie: cutii și adrese	7
5 Valori implicite și inițializare	7
5.1 Variabile locale	8
5.2 Date membre	8
6 Literali	8
6.1 Baze de numerație	8
7 Clase wrapper și autoboxing	9
7.1 Autoboxing și unboxing	9
7.2 Conversii din String	9
8 Operatori	9
8.1 Operatori aritmetici	9
8.2 Incrementare și decrementare	9
8.3 Operatori relaționali	10
8.4 Operatori logici	10
8.5 Scurtcircuitare	10
8.6 Operatorul ternar	10
8.7 Operatorul instanceof	10

9 Instrucțiuni de control	11
9.1 if - else	11
9.2 switch	11
9.3 while și do-while	11
9.4 for clasic	11
9.5 enhanced for	12
10 Clase și obiecte - baza OOP	12
10.1 Analogie	12
11 Modificatori de acces	12
12 Metode	13
12.1 Parametrii se transmit prin valoare	13
12.1.1 Exemplu cu primitiv	13
12.1.2 Exemplu cu tablou	14
12.1.3 Capcană	14
13 this	14
14 Constructori	15
14.1 Reguli	15
14.2 Constructor fără parametri	15
14.3 Constructor cu parametri	15
14.4 Apelarea unui constructor din alt constructor	15
15 static	16
15.1 Analogie	16
15.2 Metode statice	16
16 final	17
16.1 Variabilă primitivă final	17
16.2 Referință finală	17
16.3 Metodă finală	17
16.4 Clasă finală	17
17 Referințe: cel mai important concept de bază	17
17.1 Explicație	17
17.2 Analogie	18

18 Shallow copy și deep copy	18
18.1 Shallow copy	18
18.2 Deep copy	18
18.3 Cum reții	19
19 Tablouri	19
19.1 Inițializare directă	19
19.2 length	19
19.3 Indexare	19
20 Tablouri bidimensionale	19
21 Clasa Arrays	20
21.1 toString	20
21.2 deepToString	20
21.3 equals vs deepEquals	20
21.4 mismatch	20
21.5 compare	20
21.6 copyOf	21
21.7 sort și binarySearch	21
22 Citirea cu Scanner	21
22.1 Metode utile	21
22.2 Capcana nextInt + nextLine	21
23 String - noțiuni de bază	22
23.1 String este imutabil	22
23.2 == vs equals	22
24 StringBuilder și StringBuffer	22
24.1 Analogie	23
25 Comentarii și stil	23
26 Erori de compilare vs excepții la rulare	23
26.1 Eroare de compilare	23
26.2 Excepție la rulare	23
27 Mini-recapitulare: ce trebuie să poți face după Capitolul 0	23

28 Capcane frecvente de examen	24
29 Exerciții rapide	24
30 Fișă finală de memorat	25

Cum folosești documentul

Acest document este gândit ca o recapitulare de bază. Nu trebuie memorat mecanic. Pentru examen, important este să poți urmări cod Java și să anticipezi:

- ce se compilează și ce nu;
- ce se afișează;
- unde apare o excepție;
- când se creează un obiect nou și când se copiază doar o referință;
- ce se întâmplă cu variabilele statice, finale, locale și de instanță.

De reținut: Java este strict: multe greșeli se văd la compilare, iar restul apar la rulare sub formă de excepții.

1 Imaginea de ansamblu: ce este Java

Java este un limbaj de programare orientat pe obiecte. Ideea centrală este că un program Java este format din clase, iar obiectele sunt instanțe ale claselor.

1.1 Compilare și rulare

Fluxul simplificat este:

`fișier.java` $\longrightarrow$ `javac` $\longrightarrow$ `fișier.class` $\longrightarrow$ `java` $\longrightarrow$ program rulat

Codul Java nu este transformat direct în cod mașină specific procesorului, ci în **bytecode**. Bytecode-ul este executat de **JVM** - Java Virtual Machine.

1.2 JDK, JRE, JVM

Termen	Ce înseamnă
JVM	Mașina virtuală care execută bytecode-ul. Este ca un translator care știe să ruleze fișiere <code>.class</code> .
JRE	Mediul necesar pentru rularea aplicațiilor Java: JVM + biblioteci standard.
JDK	Kitul pentru dezvoltare: include JRE + instrumente precum <code>javac</code> .

1.3 Analogie ușor de reținut

Gândește-te la Java ca la un teatru:

- codul `.java` = scenariul scris de tine;
- `javac` = regizorul care transformă scenariul în instrucțiuni clare;
- bytecode-ul `.class` = scenariul tehnic;
- JVM = scena pe care se joacă piesa, indiferent de calculator.

De aici vine ideea *Write Once, Run Anywhere*: scrii o dată codul, iar JVM-ul potrivit îl poate rula pe sisteme diferite.

2 Structura minimă a unui program Java

Un program simplu:

```
public class Test {
    public static void main(String[] args) {
        System.out.println("Salut, Java!");
    }
}
```

2.1 Ce înseamnă fiecare parte

Fragment	Explicație
<code>public class Test</code>	Declară o clasă publică numită <code>Test</code> . Dacă este publică, fișierul trebuie să se numească <code>Test.java</code> .
<code>main</code>	Punctul de pornire al aplicației. De aici începe execuția.
<code>public</code>	Metoda poate fi apelată din exterior. JVM trebuie să o poată găsi.
<code>static</code>	Metoda aparține clasei, nu unui obiect. JVM o poate apela fără să creeze obiect.
<code>void</code>	Metoda nu returnează nimic.
<code>String[] args</code>	Argumente primite din linia de comandă.
<code>System.out.println</code>	Afișează text și trece la linie nouă.

Capcană de examen: Dacă numele clasei publice este `Test`, fișierul trebuie să fie `Test.java`. Dacă nu, apare eroare de compilare.

3 Pachete și importuri

Un pachet organizează clasele. Este ca un folder logic.

```
package ro.unibuc.exemplu;

import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int x = sc.nextInt();
        System.out.println(x);
    }
}
```

3.1 Reguli importante

- `package` se scrie primul în fișier, dacă există.
- `import` vine după `package`.
- Pachetul `java.lang` este importat automat. De aceea putem folosi direct `String`, `Math`, `System`.
- `import java.util.*;` importă clasele din `java.util`, dar nu și subpachetele.

De reținut: Importul nu copiază codul clasei, ci doar îți permite să scrii numele scurt al clasei.

4 Tipuri de date în Java

În Java există două mari categorii:

1. tipuri primitive;
2. tipuri referință.

4.1 Tipuri primitive

Tipurile primitive conțin direct valoarea.

Tip	Dimensiune	Exemplu
byte	1 octet	byte b = 10;
short	2 octeți	short s = 1000;
int	4 octeți	int x = 10;
long	8 octeți	long n = 100L;
float	4 octeți	float f = 3.14f;
double	8 octeți	double d = 3.14;
char	2 octeți	char c = 'A';
boolean	depinde de JVM	boolean ok = true;

4.2 Tipuri referință

Tipurile referință nu conțin direct obiectul, ci o referință către obiectul aflat în heap.

Exemple de tipuri referință:

- clase: `String`, `Scanner`, `Persoana`;
- tablouri: `int[]`, `String[]`;
- interfețe: `Runnable`, `Comparable`;
- enumerări: `enum Zi`;
- `null` - valoare specială care înseamnă „nu indică niciun obiect”.

4.3 Analogie: cutii și adrese

- Variabilă primitivă = cutie care conține valoarea direct.
- Variabilă referință = bilețel cu adresa unei cutii din depozit.
- Heap = depozitul unde stau obiectele.
- Stack = masa de lucru unde ai variabile locale și referințe.

```
int x = 10;
String s = new String("Java");
```

Aici `x` conține direct 10. Variabila `s` conține o referință către obiectul `String`.

5 Valori implicite și inițializare

5.1 Variabile locale

Variabilele locale trebuie inițializate înainte de utilizare.

```
public class Test {
    public static void main(String[] args) {
        int x;
        // System.out.println(x); // eroare de compilare
    }
}
```

5.2 Date membre

Datele membre primesc valori implicite.

```
class A {
    int x;           // 0
    double d;        // 0.0
    boolean ok;      // false
    String s;        // null
}
```

Tip	Valoare implicită
numeric	0 sau 0.0
char	caracterul cu codul 0
boolean	false
referință	null

Capcană de examen: O variabilă locală neinițializată produce eroare de compilare, nu valoare implicită.

6 Literali

Literalul este o valoare scrisă direct în cod.

```
int a = 10;           // literal int
long b = 10L;         // literal long
float c = 3.14f;      // literal float
double d = 3.14;      // literal double
char e = 'A';         // literal char
String s = "abc";     // literal String
boolean ok = true;
Object ob = null;
```

6.1 Baze de numerație

```
int a = 10;           // baza 10
int b = 0b1010;       // baza 2, adica 10
int c = 012;          // baza 8, adica 10
int d = 0xA;          // baza 16, adica 10
```

Capcană de examen: 012 nu este doisprezece în baza 10, ci 10 în baza 10, pentru că prefixul 0 indică baza 8.

7 Clase wrapper și autoboxing

Fiecare tip primitiv are o clasă asociată:

Primitiv	Wrapper
int	Integer
double	Double
char	Character
boolean	Boolean
long	Long
float	Float
byte	Byte
short	Short

7.1 Autoboxing și unboxing

```
Integer a = 10;    // autoboxing: int -> Integer
int b = a;         // unboxing: Integer -> int
```

7.2 Conversii din String

```
int x = Integer.parseInt("123");
double d = Double.parseDouble("3.14");
```

Atenție: Dacă textul nu este un număr valid, apare `NumberFormatException`.

```
int x = Integer.parseInt("12a"); // NumberFormatException
```

8 Operatori

8.1 Operatori aritmetici

```
int a = 7, b = 2;
System.out.println(a + b); // 9
System.out.println(a - b); // 5
System.out.println(a * b); // 14
System.out.println(a / b); // 3, nu 3.5
System.out.println(a % b); // 1
```

Capcană de examen: Împărțirea între doi `int` produce tot `int`.

```
System.out.println(7 / 2);    // 3
System.out.println(7 / 2.0); // 3.5
```

8.2 Incrementare și decrementare

```
int x = 5;
System.out.println(x++); // afiseaza 5, apoi x devine 6
System.out.println(++x); // x devine 7, apoi afiseaza 7
```

- `x++` = folosește valoarea curentă, apoi crește.
- `++x` = crește, apoi folosește valoarea nouă.

8.3 Operatori relaționali

```
int x = 10;
System.out.println(x > 5);    // true
System.out.println(x == 10); // true
System.out.println(x != 3);  // true
```

8.4 Operatori logici

```
boolean a = true;
boolean b = false;
System.out.println(a && b); // false
System.out.println(a || b); // true
System.out.println(!a);    // false
```

8.5 Scurtcircuitare

```
int x = 0;
if (x != 0 && 10 / x > 1) {
    System.out.println("ok");
}
```

Nu apare excepție, deoarece la `&&`, dacă prima condiție este falsă, a doua nu se mai evaluează.

```
int x = 0;
if (x != 0 & 10 / x > 1) { // aici se evalueaza ambele
    System.out.println("ok");
}
```

Aici apare `ArithmeticException`, pentru că `&` nu scurtcircuitează.

8.6 Operatorul ternar

```
int x = 10;
String rezultat = x % 2 == 0 ? "par" : "impar";
System.out.println(rezultat); // par
```

Forma generală:

`conditie ? valoareDacaTrue : valoareDacaFalse`

8.7 Operatorul instanceof

```
Object ob = "Java";
System.out.println(ob instanceof String); // true
System.out.println(ob instanceof Integer); // false
```

De reținut: `instanceof` verifică tipul real al obiectului, nu doar tipul declarat al referinței.

9 Instrucțiuni de control

9.1 if - else

```
int nota = 8;
if (nota >= 5) {
    System.out.println("promovat");
} else {
    System.out.println("nepromovat");
}
```

9.2 switch

```
int zi = 2;
switch (zi) {
    case 1:
        System.out.println("luni");
        break;
    case 2:
        System.out.println("marti");
        break;
    default:
        System.out.println("alta zi");
}
```

Capcană de examen: Dacă uiți `break`, execuția continuă în următorul `case`. Se numește *fall-through*.

9.3 while și do-while

```
int i = 0;
while (i < 3) {
    System.out.println(i);
    i++;
}
```

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 3);
```

Diferența: `do-while` execută corpul cel puțin o dată.

9.4 for clasic

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

9.5 enhanced for

```
int[] v = {10, 20, 30};
for (int x : v) {
    System.out.println(x);
}
```

Atenție: În enhanced for primești valoarea elementului, nu indexul.

```
int[] v = {1, 2, 3};
for (int x : v) {
    x = 100;
}
System.out.println(Arrays.toString(v)); // [1, 2, 3]
```

Pentru primitive, modificarea lui x nu modifică tabloul.

10 Clase și obiecte - baza OOP

O clasă este un șablon. Un obiect este o instanță a clasei.

```
class Persoana {
    String nume;
    int varsta;

    void afiseaza() {
        System.out.println(nume + " " + varsta);
    }
}

public class Test {
    public static void main(String[] args) {
        Persoana p = new Persoana();
        p.nume = "Ana";
        p.varsta = 20;
        p.afiseaza();
    }
}
```

10.1 Analogie

- Clasa = formular gol / tipar.
- Obiectul = formular completat.
- Datele membre = informațiile din formular.
- Metodele = acțiunile pe care le poate face obiectul.

11 Modificatori de acces

Modificator	Acces
public	Accesibil de oriunde.
private	Accesibil doar în aceeași clasă.

<code>protected</code>	Accesibil în același pachet și în subclase.
<code>implicit / default</code>	Accesibil doar în același pachet.

De reținut: Pentru încapsulare, datele membre se declară de obicei `private`, iar accesul se face prin metode publice de tip `get/set`.

```
class Cont {
    private double sold;

    public double getSold() {
        return sold;
    }

    public void depune(double suma) {
        if (suma > 0) {
            sold += suma;
        }
    }
}
```

Aici nu poți seta soldul direct cu o valoare negativă din exterior.

12 Metode

O metodă este o acțiune definită într-o clasă.

```
class Calculator {
    int suma(int a, int b) {
        return a + b;
    }
}
```

Forma generală:

```
[modificatori] tipReturnat numeMetoda(parametri)
    ...
```

12.1 Parametrii se transmit prin valoare

În Java, totul se transmite prin valoare. La primitive se copiază valoarea. La referințe se copiază referința.

12.1.1 Exemplu cu primitiv

```
static void modifica(int x) {
    x = 100;
}

public static void main(String[] args) {
    int a = 5;
    modifica(a);
    System.out.println(a); // 5
}
```

Metoda primește o copie a lui `a`. De aceea `a` nu se schimbă.

12.1.2 Exemplu cu tablou

```
static void modifica(int[] v) {
    v[0] = 100;
}

public static void main(String[] args) {
    int[] a = {1, 2, 3};
    modifica(a);
    System.out.println(Arrays.toString(a)); // [100, 2, 3]
}
```

De ce se modifică? Pentru că metoda primește o copie a referinței, dar copia indică spre același tablou.

12.1.3 Capcană

```
static void modifica(int[] v) {
    v[0] = 100;
    v = new int[]{7, 8, 9};
    v[1] = 200;
}

public static void main(String[] args) {
    int[] a = {1, 2, 3};
    modifica(a);
    System.out.println(Arrays.toString(a)); // [100, 2, 3]
}
```

Explicație:

- `v[0] = 100` modifică obiectul comun;
- `v = new int[]{...}` schimbă doar copia referinței locale;
- referința `a` din `main` rămâne spre tabloul inițial.

13 this

`this` este referința către obiectul curent.

```
class Persoana {
    private String nume;

    public void setNume(String nume) {
        this.nume = nume;
    }
}
```

Aici avem două variabile cu același nume:

- `nume` = parametrul metodei;
- `this.nume` = data membră a obiectului curent.

De reținut: `this` înseamnă „obiectul acesta”.

14 Constructori

Constructorul inițializează obiectul.

```
class Persoana {  
    private String nume;  
    private int varsta;  
  
    public Persoana(String nume, int varsta) {  
        this.nume = nume;  
        this.varsta = varsta;  
    }  
}
```

14.1 Reguli

- Constructorul are același nume ca clasa.
- Constructorul nu are tip returnat, nici măcar `void`.
- Poți avea mai mulți constructori prin supraîncărcare.
- Dacă nu scrii niciun constructor, compilatorul generează unul fără parametri.
- Dacă scrii măcar un constructor, compilatorul nu mai generează automat constructorul fără parametri.

14.2 Constructor fără parametri

```
class A {  
    int x;  
  
    public A() {  
        x = 10;  
    }  
}
```

14.3 Constructor cu parametri

```
class A {  
    int x;  
  
    public A(int x) {  
        this.x = x;  
    }  
}
```

14.4 Apelarea unui constructor din alt constructor

```
class Persoana {  
    String nume;  
    int varsta;  
  
    public Persoana() {  
        this("Necunoscut", 0);  
    }  
}
```



```
public Persoana(String nume, int varsta) {
    this.nume = nume;
    this.varsta = varsta;
}
}
```

Atenție: `this(...)` trebuie să fie prima instrucțiune din constructor.

15 static

`static` înseamnă că membrul aparține clasei, nu fiecărui obiect separat.

```
class Student {
    String nume;
    static int nrStudenti = 0;

    public Student(String nume) {
        this.nume = nume;
        nrStudenti++;
    }
}
```

```
Student a = new Student("Ana");
Student b = new Student("Ion");
System.out.println(Student.nrStudenti); // 2
```

15.1 Analogie

- Dată membră normală = caiet personal al fiecărui student.
- Dată statică = panou comun al clasei, văzut de toate obiectele.

15.2 Metode statice

```
class Matematica {
    static int patrat(int x) {
        return x * x;
    }
}
```

```
System.out.println(Matematica.patrat(5)); // 25
```

Atenție: O metodă statică nu poate accesa direct date membre nestatice, pentru că nu aparține unui obiect concret.

```
class A {
    int x = 10;

    static void f() {
        // System.out.println(x); // eroare
    }
}
```

16 final

`final` înseamnă „nu mai poate fi schimbat” într-un anumit sens.

16.1 Variabilă primitivă final

```
final int x = 10;
// x = 20; // eroare
```

16.2 Referință finală

```
final int[] v = {1, 2, 3};
v[0] = 100;           // corect
// v = new int[5];    // eroare
```

De reținut: La referințe, `final` blochează schimbarea adresei, nu modificarea obiectului.

16.3 Metodă finală

O metodă `final` nu poate fi suprascrisă în subclase.

16.4 Clasă finală

O clasă `final` nu poate fi extinsă.

```
final class A {}
// class B extends A {} // eroare
```

17 Referințe: cel mai important concept de bază

```
class A {
    int x;
}

public class Test {
    public static void main(String[] args) {
        A a = new A();
        a.x = 10;

        A b = a;
        b.x = 20;

        System.out.println(a.x); // 20
    }
}
```

17.1 Explicație

- `new A()` creează un obiect în heap.

- `a` primește adresa obiectului.
- `b = a` copiază adresa, nu obiectul.
- `a` și `b` indică același obiect.

17.2 Analogie

Dacă `a` are adresa unei case și îi dai aceeași adresă lui `b`, nu ai construit o a doua casă. Amândoi mergeți la aceeași casă.

Capcană de examen: Când vezi `b = a`, întreabă-te: „s-a copiat obiectul sau doar referința?”. Răspuns: doar referința.

18 Shallow copy și deep copy

18.1 Shallow copy

Shallow copy copiază câmpurile, dar pentru câmpurile de tip referință copiază doar referința.

```
class Persoana {
    String nume;
    int[] note;

    public Persoana(Persoana other) {
        this.nume = other.nume;
        this.note = other.note; // shallow copy
    }
}
```

Problema:

```
Persoana p1 = new Persoana();
p1.note = new int[]{10, 9};

Persoana p2 = new Persoana(p1);
p2.note[0] = 5;

System.out.println(p1.note[0]); // 5
```

18.2 Deep copy

Deep copy creează copii separate pentru obiectele interne mutabile.

```
class Persoana {
    String nume;
    int[] note;

    public Persoana(Persoana other) {
        this.nume = other.nume;
        this.note = Arrays.copyOf(other.note, other.note.length);
    }
}
```

Acum `p1.note` și `p2.note` sunt tablouri diferite.

18.3 Cum reții

- Shallow copy = xerox la foaia cu adrese.
- Deep copy = xerox și la foaie, și la toate documentele indicate de adrese.

19 Tablouri

Tablourile sunt obiecte, deci variabilele de tip tablou sunt referințe.

```
int[] v = new int[3];  
v[0] = 10;  
v[1] = 20;  
v[2] = 30;
```

19.1 Inițializare directă

```
int[] v = {10, 20, 30};  
String[] nume = {"Ana", "Ion"};
```

19.2 length

```
int[] v = {10, 20, 30};  
System.out.println(v.length); // 3
```

Atenție: La tablou este `length`, nu `length()`.

19.3 Indexare

Indicii încep de la 0.

```
int[] v = {10, 20, 30};  
System.out.println(v[0]); // 10  
System.out.println(v[2]); // 30  
// System.out.println(v[3]); // ArrayIndexOutOfBoundsException
```

20 Tablouri bidimensionale

În Java, un tablou bidimensional este un tablou de tablouri.

```
int[][] a = {  
    {1, 2, 3},  
    {4, 5},  
    {6}  
};
```

Liniile pot avea lungimi diferite.

```
for (int i = 0; i < a.length; i++) {  
    for (int j = 0; j < a[i].length; j++) {  
        System.out.print(a[i][j] + " ");  
    }  
}
```

```
    }  
    System.out.println();  
}
```

Capcană de examen: Pentru numărul de coloane pe linia `i`, folosești `a[i].length`, nu `a.length`.

21 Clasa Arrays

Pentru a folosi clasa `Arrays`:

```
import java.util.Arrays;
```

21.1 toString

```
int[] v = {1, 2, 3};  
System.out.println(v); // ceva de forma [I@...  
System.out.println(Arrays.toString(v)); // [1, 2, 3]
```

21.2 deepToString

```
int[][] a = {{1, 2}, {3, 4}};  
System.out.println(Arrays.toString(a)); // referinte ale liniilor  
System.out.println(Arrays.deepToString(a)); // [[1, 2], [3, 4]]
```

21.3 equals vs deepEquals

```
int[][] a = {{1, 2}, {3, 4}};  
int[][] b = {{1, 2}, {3, 4}};  
  
System.out.println(Arrays.equals(a, b)); // false  
System.out.println(Arrays.deepEquals(a, b)); // true
```

21.4 mismatch

```
int[] a = {1, 2, 3};  
int[] b = {1, 2, 4};  
System.out.println(Arrays.mismatch(a, b)); // 2
```

Returnează primul index unde diferă tablourile sau `-1` dacă sunt egale.

21.5 compare

```
int[] a = {1, 2, 3};  
int[] b = {1, 2, 4};  
System.out.println(Arrays.compare(a, b)); // negativ
```

Compară lexicografic, ca într-un dicționar.

21.6 copyOf

```
int[] a = {1, 2, 3};
int[] b = Arrays.copyOf(a, 5);
System.out.println(Arrays.toString(b)); // [1, 2, 3, 0, 0]
```

21.7 sort și binarySearch

```
int[] v = {5, 1, 3};
Arrays.sort(v);
System.out.println(Arrays.toString(v)); // [1, 3, 5]

System.out.println(Arrays.binarySearch(v, 3)); // 1
```

Atenție: `binarySearch` presupune că tabloul este sortat.

22 Citirea cu Scanner

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int x = sc.nextInt();
        double d = sc.nextDouble();
        String s = sc.next();
        System.out.println(x + " " + d + " " + s);
    }
}
```

22.1 Metode utile

Metodă	Ce citește
<code>nextInt()</code>	un număr întreg
<code>nextDouble()</code>	un număr real
<code>next()</code>	un cuvânt fără spații
<code>nextLine()</code>	o linie întreagă
<code>hasNext()</code>	verifică dacă mai există token-uri
<code>hasNextInt()</code>	verifică dacă următorul token este int

22.2 Capcana `nextInt()` + `nextLine()`

```
Scanner sc = new Scanner(System.in);
int x = sc.nextInt();
String linie = sc.nextLine();
```

După `nextInt()`, caracterul de linie nouă rămâne în buffer, deci `nextLine()` poate citi o linie goală.

Soluție:

```
int x = sc.nextInt();
sc.nextLine(); // consumam restul liniei
String linie = sc.nextLine();
```

23 String - noțiuni de bază

String este o clasă, dar se folosește foarte des ca un tip de bază.

```
String s = "Java";
System.out.println(s.length()); // 4
System.out.println(s.charAt(0)); // J
System.out.println(s.substring(1)); // ava
```

23.1 String este imutabil

```
String s = "java";
s.toUpperCase();
System.out.println(s); // java
```

Metoda `toUpperCase()` creează un String nou, dar nu schimbă obiectul vechi.

Corect:

```
s = s.toUpperCase();
System.out.println(s); // JAVA
```

23.2 == vs equals

```
String a = "java";
String b = "java";
String c = new String("java");

System.out.println(a == b); // true
System.out.println(a == c); // false
System.out.println(a.equals(c)); // true
```

- `==` compară referințele.
- `equals` compară conținutul.

Capcană de examen: Pentru String-uri, aproape mereu compari cu `equals`, nu cu `==`.

24 StringBuilder și StringBuffer

Dacă trebuie să modifice des un șir, folosești `StringBuilder`.

```
StringBuilder sb = new StringBuilder();
sb.append("Java");
sb.append(" ");
sb.append("PAO");
System.out.println(sb.toString()); // Java PAO
```

24.1 Analogie

- **String** = text tipărit pe hârtie; dacă vrei să schimbi ceva, printezi altă foaie.
- **StringBuilder** = tablă albă; poți șterge și modifica direct.
- **StringBuffer** = tablă albă cu lacăt; este sincronizat pentru fire de execuție.

25 Comentarii și stil

```
// comentariu pe o linie

/*
  comentariu
  pe mai multe linii
*/
```

Pentru examen, codul trebuie să fie clar, nu neapărat sofisticat. Scrie nume de variabile ușor de urmărit.

26 Erori de compilare vs excepții la rulare

26.1 Eroare de compilare

Apare înainte să rulezi programul.

```
int x = "abc"; // eroare de compilare
```

26.2 Excepție la rulare

Programul se compilează, dar se oprește la rulare.

```
int x = 10 / 0; // ArithmeticException
```

27 Mini-recapitulare: ce trebuie să poți face după Capitolul 0

După acest capitol trebuie să poți:

1. să explici diferența dintre JDK, JRE și JVM;
2. să scrii structura minimă a unui program Java;
3. să diferențiezi tipurile primitive de referințe;
4. să știi valorile implicite pentru date membre;
5. să urmărești expresii cu operatori, inclusiv `++x` și `x++`;
6. să citești cod cu `if`, `switch`, `for`, `while`;
7. să explici ce face `this`;
8. să scrii constructori;
9. să explici `static` și `final`;
10. să urmărești efectele unei atribuirii între referințe;
11. să faci diferența dintre shallow copy și deep copy;

12. să lucrezi cu tablouri și metode din **Arrays**;
13. să citești date cu **Scanner**;
14. să explici de ce **String** este imutabil;
15. să alegi între **String**, **StringBuilder** și **StringBuffer**.

28 Capcane frecvente de examen

Capcană	Răspuns corect
<code>b = a</code> la obiecte	Se copiază referința, nu obiectul.
Variabilă locală neinițializată	Eroare de compilare.
Dată membră neinițializată	Primește valoare implicită.
<code>7 / 2</code>	Rezultatul este 3, nu 3.5.
<code>x++</code>	Folosește valoarea veche, apoi incrementează.
<code>++x</code>	Incrementează, apoi folosește valoarea nouă.
<code>String.toUpperCase()</code>	Nu modifică stringul inițial. Returnează alt <code>String</code> .
<code>==</code> la obiecte	Compară referințe.
<code>equals</code> la <code>String</code>	Compară conținutul.
<code>v.length</code>	La tablou este câmp, nu metodă.
<code>s.length()</code>	La <code>String</code> este metodă.
<code>Arrays.equals</code> pe 2D	Compară referințele liniilor. Pentru conținut: <code>deepEquals</code> .
<code>final int[] v</code>	Nu poți schimba referința, dar poți modifica elementele.
<code>nextInt()</code> urmat de <code>nextLine()</code>	Trebuie consumată linia rămasă.

29 Exerciții rapide

Exercițiul 1

Ce se afișează?

```
int x = 5;
System.out.println(x++);
System.out.println(++x);
```

Răspuns:

```
5
7
```

Exercițiul 2

Ce se afișează?

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 100;
System.out.println(a[0]);
```

Răspuns: 100. `a` și `b` indică același tablou.

Exercițiul 3

Ce se afișează?

```
String s = "java";  
s.toUpperCase();  
System.out.println(s);
```

Răspuns: java. String-ul inițial nu se modifică.

Exercițiul 4

Ce se afișează?

```
String a = "abc";  
String b = new String("abc");  
System.out.println(a == b);  
System.out.println(a.equals(b));
```

Răspuns:

```
false  
true
```

Exercițiul 5

Ce se afișează?

```
int[] [] a = {{1, 2}, {3}};  
System.out.println(a.length);  
System.out.println(a[0].length);  
System.out.println(a[1].length);
```

Răspuns:

```
2  
2  
1
```

30 Fișă finală de memorat

- Java rulează bytecode pe JVM.
- Un program pornește din `public static void main(String[] args)`.
- Primitivele țin valoarea; referințele țin adresa obiectului.
- `null` înseamnă că referința nu indică niciun obiect.
- Obiectele și tablourile sunt în heap.
- Variabilele locale trebuie inițializate.
- Datele membre primesc valori implicite.
- Parametrii se transmit prin valoare.
- La referințe, se copiază adresa, nu obiectul.
- `static` aparține clasei.
- `final` blochează schimbarea variabilei/referinței, dar nu face automat obiectul imutabil.

- `String` este imutabil.
- `StringBuilder` este mutabil.
- `==` compară referințe la obiecte.
- `equals()` compară conținutul dacă este redefinit corespunzător.
- Tablourile au `length`, String-urile au `length()`.
- Pentru tablouri 2D folosești `deepToString` și `deepEquals`.